

CAHIER DES CHARGES

Application de r  servation de ressources partag  es

Tiers-lieu L'Escale (organisation fictive)



Formation ouverte aux personnes en situation de handicap "sous r  serve que la situation de sant   de la personne soit stabilis  e, de l'absence de contre-indications    l'exercice de la profession et dans la limite d'am  nagements raisonnables" (loi du 11 f  vrier 2005)

Pour plus d'informations : contacter notre r  f  rent handicap

entretien@lasallebourges.com

Version du document : Version 1.0

Date : 07/10/2026

TABLE DES MATIERES

1. Présentation du projet.....	3
1.1 Objet du document	3
1.2 Contexte	3
1.3 Objectifs pédagogiques	3
1.4 Rôle du formateur	3
2. Périmètre	4
2.1 Ce qui est inclus.....	4
2.2 Ce qui est exclu	4
2.3 Les bonus	4
3. Acteurs et droits	4
4. Exigences fonctionnelles	5
5. Règles de gestion.....	6
6. Exigences non fonctionnelles	7
6.1 Sécurité	7
6.2 Données personnelles.....	7
6.3 Accessibilité	7
6.4 Ergonomie	7
6.5 Qualité et maintenabilité	7
7. Contraintes techniques.....	8
7.1 Choix imposés	8
7.2 Architecture en couches.....	8
8. Organisation du projet.....	9
9. Planning	9
10. Livrables.....	10
11. Évaluation	11
11.1 Modalités.....	11
11.2 Compétences évaluées	12
12. Documents de référence.....	13

1. PRESENTATION DU PROJET

1.1 Objet du document

Ce cahier des charges encadre le projet du module full stack de troisième année. Il reprend la fiche d'expression de besoin du tiers-lieu L'Escale, fournie séparément, et la traduit en exigences fonctionnelles, non fonctionnelles et techniques. Il précise aussi l'organisation du projet, le planning, les livrables attendus et les modalités d'évaluation.

La fiche d'expression de besoin décrit ce que veut le client. Ce cahier des charges décrit ce que vous devez livrer et dans quelles conditions. Les deux documents doivent être lus ensemble.

1.2 Contexte

L'Escale est un tiers-lieu associatif fictif d'environ 400 adhérents. Il met à leur disposition des salles de réunion, un studio d'enregistrement, un véhicule utilitaire et une trentaine d'équipements empruntables. Les réservations sont aujourd'hui gérées par e-mail et dans un tableur partagé, ce qui provoque des doubles réservations, une charge importante pour l'équipe et une absence de traçabilité.

Le tiers-lieu souhaite une application web qui permette aux adhérents de réserver eux-mêmes les ressources, qui applique automatiquement ses règles d'utilisation et qui donne à l'équipe une vision claire de l'activité.

1.3 Objectifs pédagogiques

Le projet est réalisé individuellement. Il a pour but de mobiliser l'ensemble des compétences des trois blocs du titre professionnel Concepteur développeur d'applications (RNCP37873) : développer une application sécurisée, la concevoir en couches, puis préparer son déploiement. La compétence transversale d'anglais technique est travaillée lors du tutoriel officiel Next.js réalisé en début de module. Le projet couvre ainsi les onze compétences professionnelles du titre ; les preuves attendues pour chacune sont détaillées en section 11.2.

1.4 Rôle du formateur

Pendant tout le projet, le formateur joue le rôle du client. La fiche d'expression de besoin ne répond pas à toutes les questions : c'est à vous d'identifier les points flous et de les poser au client plutôt que de faire des suppositions. Les réponses sont données individuellement. Toutes les questions posées et les réponses obtenues doivent être consignées dans votre dossier d'analyse.

Si vous faites malgré tout une hypothèse, elle doit être écrite et justifiée dans le dossier d'analyse.

2. PERIMETRE

2.1 Ce qui est inclus

Le projet couvre l'ensemble des besoins décrits dans la fiche d'expression de besoin :

- la consultation et la recherche des ressources ;
- la réservation et son annulation ;
- la validation de certaines demandes par un gestionnaire ;
- la gestion du catalogue et des périodes de maintenance ;
- les notifications dans l'application ;
- le journal d'activité ;
- gestion des comptes par l'administrateur.

2.2 Ce qui est exclu

Conformément à la fiche d'expression de besoin, les éléments suivants ne font pas partie du projet : le paiement en ligne et la gestion des cotisations, une application mobile native, la gestion des clés et des badges d'accès, et l'inscription libre des adhérents. Les comptes sont créés par l'administrateur ou importés à partir d'un jeu de données.

2.3 Les bonus

Les besoins classés « souhaitables » par le client (notifications par e-mail, export dans un agenda personnel) ne sont à traiter qu'une fois toutes les exigences indispensables et importantes livrées et testées. Il en va de même pour toute amélioration technique non demandée, comme une mise en cache ou un outil de supervision. Un bonus ne compense jamais une exigence indispensable manquante. Pour tenir le planning, le tableau de bord des gestionnaires (EF-13), classé important par le client, est également traité comme un bonus.

3. ACTEURS ET DROITS

L'application distingue quatre profils. Le tableau ci-dessous résume les grandes familles d'actions autorisées pour chacun. Il ne remplace pas votre propre analyse des cas d'utilisation.

Action	Visiteur	Adhérent	Gestionnaire	Administrateur
Consulter et rechercher dans le catalogue	Oui	Oui	Oui	Oui
Réserver une ressource, consulter et annuler ses réservations	Non	Oui	Oui	Oui
Consulter ses notifications	Non	Oui	Oui	Oui
Consulter ses données personnelles et supprimer son compte	Non	Oui	Oui	Oui
Valider ou refuser une demande de réservation	Non	Non	Oui	Oui

Action	Visiteur	Adhérent	Gestionnaire	Administrateur
Gérer le catalogue et les périodes de maintenance	Non	Non	Oui	Oui
Consulter le journal d'activité et le tableau de bord	Non	Non	Oui	Oui
Gérer les comptes et attribuer les rôles	Non	Non	Non	Oui

4. EXIGENCES FONCTIONNELLES

Chaque exigence fait référence au besoin correspondant de la fiche d'expression de besoin (colonne « Réf. FEB »). Les exigences indispensables doivent toutes être livrées. Les exigences importantes sont attendues dans la version finale. Les exigences souhaitables sont des bonus.

Code	Exigence	Réf. FEB	Priorité
EF-01	L'utilisateur peut se connecter et se déconnecter. Il n'existe pas d'inscription libre.	Section 9 – Hors périmètre	Indispensable
EF-02	L'administrateur peut créer un compte, attribuer ou retirer le rôle de gestionnaire et désactiver un compte (règle R7).	B10	Indispensable
EF-03	L'utilisateur connecté peut consulter les données personnelles le concernant et supprimer son compte.	Section 7 – Nos autres exigences	Indispensable
EF-04	Toute personne, connectée ou non, peut consulter le catalogue des ressources et la fiche détaillée de chacune (description, photo, catégorie, conditions d'utilisation).	B01	Indispensable
EF-05	L'utilisateur peut filtrer le catalogue par catégorie et par mot-clé, et n'afficher que les ressources disponibles sur une période donnée.	B02	Indispensable
EF-06	L'adhérent peut réserver une ressource sur un créneau. L'application refuse toute réservation qui enfreint une règle de gestion ou qui chevauche une réservation existante, y compris en cas de demandes simultanées, et explique la raison du refus.	B03, B04	Indispensable
EF-07	L'adhérent peut consulter ses réservations à venir et passées, et annuler une réservation dans les conditions de la règle R4.	B03	Indispensable
EF-08	Pour les ressources qui le nécessitent, une réservation est d'abord une demande en attente. Un gestionnaire la valide ou la refuse avec un motif obligatoire (règles R6 et R8).	B05	Indispensable
EF-09	Le gestionnaire peut ajouter, modifier et retirer des ressources, photo comprise.	B06	Indispensable
EF-10	Le gestionnaire peut déclarer une période de maintenance sur une ressource (règle R5).	B06	Indispensable
EF-11	Les utilisateurs reçoivent dans l'application une notification pour chaque événement qui les concerne, et voient le nombre de notifications non lues.	B07	Importante

Code	Exigence	Réf. FEB	Priorité
EF-12	Toutes les actions importantes sont enregistrées dans un journal d'activité (auteur, action, cible, date), consultable par les gestionnaires sous forme de liste chronologique.	B08	Importante
EF-13	Le gestionnaire dispose d'un tableau de bord présentant les demandes en attente, les réservations du jour et les ressources les plus utilisées.	B09	Souhaitable
EF-14	Les notifications sont également envoyées par e-mail.	B11	Souhaitable
EF-15	L'adhérent peut exporter une réservation au format iCalendar pour l'ajouter à son agenda.	B12	Souhaitable

5. REGLES DE GESTION

Les règles de gestion sont celles définies par le client dans la fiche d'expression de besoin. Elles sont rappelées ci-dessous et doivent être appliquées côté serveur, quelle que soit l'interface utilisée. Chacune d'elles doit être couverte par au moins un test automatisé.

Règle	Description
R1 – Quota	Un adhérent ne peut pas avoir plus de trois réservations à venir par catégorie de ressource (salles, studio, véhicule, matériel). Les demandes en attente de validation comptent dans ce quota.
R2 – Durée	Une salle se réserve pour quatre heures au maximum ; le matériel et le véhicule pour trois jours au maximum.
R3 – Délai	On ne peut pas réserver plus de trente jours à l'avance.
R4 – Annulation	Un adhérent peut annuler sa réservation jusqu'à 24 heures avant son début. Au-delà, seul un gestionnaire peut l'annuler.
R5 – Maintenance	Une ressource en maintenance ne peut pas être réservée sur la période concernée. Si des réservations existaient déjà, les adhérents concernés sont prévenus.
R6 – Refus motivé	Un refus de demande doit toujours être accompagné d'un motif, visible par l'adhérent.
R7 – Compte désactivé	Un compte désactivé ne peut plus se connecter, et ses réservations à venir sont annulées.
R8 – Demande non traitée	Une demande qui n'a pas été validée ou refusée avant le début du créneau demandé est automatiquement annulée, et l'adhérent en est prévenu.

Ces règles ne couvrent pas forcément tous les cas. Repérer ceux qui manquent et les faire préciser par le client fait partie du travail d'analyse attendu.

6. EXIGENCES NON FONCTIONNELLES

6.1 Sécurité

Chaque action est autorisée ou refusée côté serveur, dans la Server Action ou le Route Handler concerné, en fonction du rôle de l'utilisateur et de la propriété des données. Masquer un bouton dans l'interface ne constitue pas une protection. Un adhérent ne doit jamais pouvoir lire ou modifier les données d'un autre adhérent, même en modifiant une URL ou une requête.

Toutes les données reçues du client sont validées avec Zod avant d'être traitées. Les mots de passe sont stockés sous forme de hash avec un algorithme adapté (bcrypt ou Argon2).

L'application est protégée contre les injections, le XSS et le CSRF, et envoie les en-têtes HTTP de sécurité usuels, dont une Content Security Policy. Aucun secret (mot de passe, clé, chaîne de connexion) n'est versionné dans le dépôt.

En fin de projet, vous remettez la checklist OWASP Top 10 renseignée. Pour chaque risque, elle indique comment l'application s'en protège ou pourquoi il ne s'applique pas.

Des tests de sécurité automatisés vérifient que ces protections fonctionnent, par exemple qu'un adhérent ne peut pas annuler la réservation d'un autre adhérent ni accéder aux pages réservées aux gestionnaires. Pendant toute la durée du projet, vous tenez aussi un journal de veille sur les vulnérabilités qui concernent les technologies utilisées (avis de sécurité, CVE, alertes sur les dépendances), en indiquant les failles éventuellement trouvées et les corrections apportées.

6.2 Données personnelles

L'application respecte le RGPD. Elle ne collecte que les données nécessaires, applique les durées de conservation fixées par le client (un an pour le journal d'activité, anonymisation des réservations passées à la suppression d'un compte) et permet à chaque utilisateur de consulter ses données et de supprimer son compte. Une page d'information explique quelles données sont collectées et pourquoi.

6.3 Accessibilité

Les parcours principaux (rechercher une ressource, réserver, annuler) doivent être utilisables au clavier seul et avec un lecteur d'écran. Sont notamment attendus une structure de titres cohérente, des libellés associés à tous les champs de formulaire, des alternatives textuelles pour les images, des contrastes suffisants et des messages d'erreur explicites. L'application vise la conformité aux WCAG 2.2 niveau AA sur les parcours principaux. Le RGAA sert de méthode de vérification.

6.4 Ergonomie

L'application est conçue pour mobile en priorité et s'adapte aux écrans d'ordinateur. Chaque action donne un retour clair à l'utilisateur : confirmation en cas de succès, explication compréhensible en cas de refus.

6.5 Qualité et maintenabilité

Le code est écrit en TypeScript en mode strict, sans erreur ESLint, et formaté avec Prettier. Les services métier ne dépendent pas de Next.js, afin de pouvoir être testés de façon isolée. Chaque

règle de gestion est couverte par au moins un test unitaire, et l'application dispose d'au moins un parcours de test de bout en bout.

Le code source est documenté : les services et les fonctions exportées comportent des commentaires JSDoc. Les conventions de nommage (fichiers, variables, tables et colonnes) sont définies dans le README et respectées dans tout le projet. La conception des composants métier doit pouvoir être justifiée au regard des bonnes pratiques de la programmation orientée objet, en particulier la séparation des responsabilités et l'injection des dépendances.

7. CONTRAINTES TECHNIQUES

7.1 Choix imposés

Les choix techniques suivants sont imposés. Lorsqu'une alternative est laissée ouverte, votre choix doit être justifié dans un document de décision d'architecture (ADR).

Domaine	Choix imposé
Langage	TypeScript, en mode strict.
Framework	Next.js avec l'App Router.
Base relationnelle	PostgreSQL, avec Prisma ou Drizzle comme ORM (choix à justifier). Le schéma évolue uniquement par migrations, et un script de seed fournit un jeu d'essai complet. Une procédure de sauvegarde et de restauration de la base est documentée et testée.
Base NoSQL	MongoDB, pour le journal d'activité et les notifications.
Authentification	Auth.js.
Validation	Zod, pour toutes les données entrantes.
Tests	Vitest pour les tests unitaires et d'intégration, Playwright pour les tests de bout en bout.
Environnement	Docker Compose : l'application et ses deux bases de données se lancent en local avec une seule commande.
Versionnement	Git et GitHub, avec une branche par fonctionnalité et une intégration par pull request.
Intégration continue	GitHub Actions : vérification du lint, des tests et du build à chaque pull request, dès la première étape.
Déploiement	Un environnement de recette accessible en ligne, déployé automatiquement depuis la branche principale. La plateforme d'hébergement est à faire valider par le formateur.

7.2 Architecture en couches

L'application est organisée en quatre couches, chacune ne pouvant appeler que la couche située juste en dessous :

- **Présentation** : les pages, layouts et composants du dossier app/ et des composants d'interface.

- **Contrôleurs** : les Server Actions et Route Handlers, qui vérifient l'authentification et les droits, valident les entrées et appellent les services.
- **Services métier** : la logique métier et les règles de gestion, indépendantes de Next.js.
- **Accès aux données** : les repositories, seuls autorisés à interroger PostgreSQL et MongoDB.

Un composant ne doit jamais interroger directement une base de données ni appeler un repository. Ce découpage est vérifié automatiquement par une règle ESLint sur les imports, exécutée dans l'intégration continue.

Le dossier de conception identifie également les besoins d'éco-conception de l'application (poids des pages et des images, nombre de requêtes, pagination des listes, mise en cache) et les choix faits pour y répondre.

8. ORGANISATION DU PROJET

Le projet est piloté avec la méthode Kanban, une méthode agile adaptée au travail individuel. Le travail est découpé en tickets (user stories ou tâches techniques), chacun accompagné de critères d'acceptation et d'une priorité. Ces tickets sont suivis dans un tableau GitHub Projects à quatre colonnes : « À faire », « En cours », « En revue » et « Terminé ». Pour éviter de disperser vos efforts, vous ne devez pas avoir plus de deux tickets « En cours » en même temps. Chaque fonctionnalité est développée sur sa propre branche et intégrée par une pull request dont l'intégration continue doit être au vert.

Le planning découpe le projet en cinq étapes et fixe trois jalons, auxquels l'avancement est vérifié. Votre tableau doit refléter à tout moment l'état réel du projet. Lors de la soutenance, vous présentez un court bilan qui compare ce qui était prévu à chaque étape avec ce qui a été réalisé, et qui explique les écarts.

Entre les deux périodes de cours, vous terminez l'étape 3 en autonomie. Ce travail représente environ 14 heures. La veille sécurité et la documentation du code se font au fil de l'eau, tout au long du projet. Pour garder le projet vivant et éviter un rattrapage de dernière minute, au moins une pull request doit être intégrée toutes les deux semaines. L'historique du dépôt sert de preuve de cette régularité.

9. PLANNING

Le module représente 63 heures de cours, réparties en cinq jours en octobre et quatre jours en janvier, auxquelles s'ajoute un travail personnel d'environ 14 heures pendant la période entre les deux cours d'octobre et janvier.

Période	Étape	Contenu	Jalon
J1 – J2	Tutoriel Next.js	Cours d'introduction, puis tutoriel officiel Next.js (chapitres 1 à 16).	

Période	Étape	Contenu	Jalon
J2 – J3	Étape 1 : analyse et conception	Analyse des besoins, user stories, maquettes, modèle de données, architecture, Docker Compose et intégration continue. Remise des dossiers d'analyse et de conception au J3.	
J3 – J5	Étape 2 : socle applicatif	Authentification et rôles, gestion du catalogue, mise en place des couches. Démonstration de 5 minutes par étudiant au J5, avec un retour du formateur.	Jalon 1 : démonstration (J5)
J5, puis période entre les deux cours d'octobre et janvier	Étape 3 : réservations	Réservations et règles de gestion, workflow de validation, plan de tests et tests unitaires. Étape terminée en autonomie pendant la période entre les deux cours d'octobre et janvier, avec les diagrammes de cas d'utilisation et de séquence et la procédure de sauvegarde et de restauration de la base.	Jalon 2 : PR « Étape 3 » mi-décembre
J6 – J7	Étape 4 : suivi de l'activité	Revue du travail réalisé pendant la période entre les deux cours d'octobre et janvier, journal d'activité et notifications (MongoDB), test de bout en bout, tests de sécurité.	
J7 – J8	Étape 5 : déploiement	Déploiement continu, mise en recette, documentation de déploiement, jeu d'essai, éco-conception, dossier sécurité. Répétition de la soutenance en binômes en fin de J8 (1 h).	
J9	Évaluation	Examen écrit (1 h), finalisation, gel du code à 12 h, soutenances l'après-midi.	Jalon 3 : tag v1.0 et soutenance

10. LIVRABLES

Tous les livrables sont versionnés dans votre dépôt GitHub. Les documents sont placés dans un dossier docs/ à la racine du dépôt.

Livrable	Contenu attendu	Échéance
Dossier d'analyse	User stories avec critères d'acceptation, cas d'utilisation, questions posées au client et réponses obtenues, hypothèses justifiées, maquettes des écrans principaux et schéma de leur enchaînement.	J3, puis tenu à jour

Livrable	Contenu attendu	Échéance
Dossier de conception	Modèle conceptuel, modèle logique et modèle physique des données, script de création de la base, diagramme de cas d'utilisation, diagrammes de séquence des cas les plus significatifs, schéma d'architecture, au moins deux documents de décision d'architecture (ADR), besoins d'éco-conception.	J3, puis tenu à jour
Code source	Application complète, migrations, script de seed, fichier Docker Compose, fichier .env.example.	Tag v1.0 au J9, 12 h
Plan et tests	Plan de tests rédigé, environnement de test dédié, tests unitaires, tests de sécurité, tests d'intégration des repositories, au moins un parcours de bout en bout (réservation puis annulation par un adhérent), jeu d'essai de la fonctionnalité la plus représentative (entrées, résultats attendus, résultats obtenus, analyse des écarts).	Tests unitaires au jalon 2, ensemble au J9
Pipeline CI/CD	Intégration continue (lint, tests, build) puis déploiement automatique en recette.	CI au J3, CD au J8
README	Présentation du projet et procédure pour l'installer et le lancer en local.	À jour au jalon 1, final au J9
Documentation de déploiement	Variables d'environnement, scripts de déploiement documentés, procédure de déploiement, procédure de retour à la version précédente (rollback), procédure d'exécution des tests d'intégration, système et d'acceptation, procédure de sauvegarde et de restauration de la base.	J9
Dossier sécurité	Checklist OWASP Top 10 renseignée et journal de veille sur les vulnérabilités de sécurité.	J9
Support de soutenance	Support de présentation pour la démonstration, l'explication des choix et le bilan prévu/réalisé.	J9

11. ÉVALUATION

11.1 Modalités

L'évaluation s'appuie sur trois éléments : le suivi aux jalons tout au long du projet, un examen écrit d'une heure construit sur le modèle du questionnaire professionnel du titre (documentation technique en anglais, questions en français et en anglais), et une soutenance individuelle.

La soutenance dure 30 minutes : 15 minutes de démonstration, de présentation de vos choix et de bilan de projet, puis 10 minutes de questions sur votre code, qui peuvent inclure une modification réalisée en direct. Les 5 dernières minutes servent à la délibération. La version évaluée est celle du tag v1.0 posé le dernier jour à 12 h ; les modifications ultérieures ne sont pas prises en compte.

La soutenance se prépare tout au long du projet. Votre présentation suit le plan ci-dessous, inspiré de celui que le référentiel du titre prévoit pour un projet réalisé en formation :

- l'expression des besoins, les objectifs et les limites du projet ;
- l'environnement technique ;
- les réalisations, compétence par compétence : maquettes et leur enchaînement, architecture, modèle de données, interfaces, composants métier et accès aux données ;
- les éléments de sécurité de l'application ;
- le plan de tests et le jeu d'essai de la fonctionnalité la plus représentative ;
- la veille sur les vulnérabilités de sécurité ;
- le bilan prévu/réalisé, avec les satisfactions et les difficultés rencontrées.

Pendant les étapes 4 et 5, collectez au fur et à mesure les captures d'écran et les extraits de code que vous présenterez. Le support s'assemble ensuite en travail personnel.

Deux temps d'entraînement à l'oral sont prévus. La démonstration du jalon 1, au J5, est un premier essai de 5 minutes par étudiant, suivi d'un court retour du formateur. En fin de J8, une répétition d'une heure se déroule en binômes : chaque étudiant présente sa soutenance à un camarade, qui lui pose des questions et lui fait un retour à l'aide d'une courte grille, puis les rôles s'inversent.

11.2 Compétences évaluées

Le tableau suivant indique, pour chaque compétence du titre, les preuves sur lesquelles elle est évaluée. L'acquisition de chaque compétence est appréciée à partir des critères d'évaluation officiels du référentiel d'évaluation du titre CDA (section 3.2).

Bloc	Compétence	Preuves attendues
BC01	Installer et configurer son environnement de travail	Environnement Docker Compose fonctionnel, configuration ESLint et Prettier, fichier .env.example, README d'installation.
BC01	Développer des interfaces utilisateur	Écrans responsives et accessibles, formulaires avec validation et messages d'erreur clairs.
BC01	Développer des composants métier	Services appliquant les règles de gestion, documentés et testés de façon isolée, tests de sécurité.
BC01	Contribuer à la gestion d'un projet informatique	Tableau Kanban GitHub Projects tenu à jour, tickets avec critères d'acceptation, historique des pull requests, respect des jalons et régularité pendant la période entre les deux cours d'octobre et janvier, bilan prévu/réalisé présenté en soutenance.
BC02	Analyser les besoins et maquetter une application	Dossier d'analyse, pertinence des questions posées au client, maquettes et schéma de leur enchaînement.
BC02	Définir l'architecture logicielle	Schéma d'architecture, ADR, respect du découpage en couches vérifié par la CI, besoins d'éco-conception.
BC02	Concevoir et mettre en place une base de données relationnelle	MCD et modèle physique, script de création, migrations, contraintes d'intégrité, jeu d'essai complet, sauvegarde et restauration testées.

Bloc	Compétence	Preuves attendues
BC02	Développer des composants d'accès aux données SQL et NoSQL	Repositories PostgreSQL et MongoDB.
BC03	Préparer et exécuter les plans de tests	Plan de tests, environnement de test, tests unitaires, de sécurité, d'intégration et de bout en bout exécutés dans la CI, jeu d'essai analysé.
BC03	Préparer et documenter le déploiement	Documentation et scripts de déploiement, procédure de rollback, procédure d'exécution des tests, veille sécurité.
BC03	Contribuer à la mise en production dans une démarche DevOps	Pipeline CI/CD fonctionnel, application déployée en recette, rapports de CI interprétés.
Transverse	Sécurité de l'application	Contrôle des droits côté serveur, validation des entrées, tests de sécurité, checklist OWASP Top 10, journal de veille.

12. DOCUMENTS DE REFERENCE

- Fiche d'expression de besoin du tiers-lieu L'Escale.
- Référentiel du titre professionnel Concepteur développeur d'applications : francecompetences.fr/recherche/rncp/37873/
- Tutoriel officiel Next.js : nextjs.org/learn
- Référentiel d'évaluation du titre CDA (TP-01281, millésime 04, mise à jour du 02/07/2024).
- OWASP Top 10 : owasp.org/Top10
- Référentiel général d'amélioration de l'accessibilité (RGAA) : accessibilite.numerique.gouv.fr